11.3.2 xy brushing

Now we'll create a brush to select some of these points. Recall when we used a brush in chapter 9 that we only allowed brushing along the x-axis. This time, we allow brushing along both x- and y-axes. Then we can drag a rectangle over any part of the canvas. In listing 11.14, you can see how quick and easy it is to add a brush to our canvas. We'll also add a function to highlight any circles in the brushed region. In this example we use `d3.scale.identity` for our `.x()` and `.y()` selectors. All `d3.scale.identity` does is create a scale where the domain and range are exactly the same. It's useful for times like these when the function operates with a scale but your scale domain directly matches the range of your graphical area.

Listing 11.14 xy brushing

```
var brush = d3.svg.brush()
  .x(d3.scale.identity().domain([0, 500]))
  .y(d3.scale.identity().domain([0, 500]))
  .on("brush", brushed);

d3.select("svg").call(brush)

function brushed() {
  var e = brush.extent();
  d3.selectAll("circle")
  .style("fill", function (d) {
    if (d.x >= e[0][0] && d.x <= e[1][0]
             && d.y >= e[0][1] && d.y <= e[1][1])
    {
      return "darkred";
    }
    else {
      return "pink";
    }
  });
};
```

Because we aren't going to adjust scale settings, we can define them inline.

Tests to see if the data is in our selected area

Colors the points in the selected area dark red

Colors the points outside the selected area pink

With this brushing code, we can now see the circles in the brushed region, as shown in figure 11.12.

This works, but it's terribly inefficient. It checks every point on the canvas without using any mechanism to ignore points that might be well outside the selection area. Finding points within a prescribed area is an old problem that has been well explored. One of the tools available to solve that problem quickly and easily is a quadtree. You may ask, what is a quadtree and what should I use it for?

A *quadtree* is a method for optimizing spatial search by dividing a plane into a series of quadrants. You then divide each of those quadrants into quadrants, until every point on that plane falls in its own quadrant. By dividing the xy plane like this, you nest the points you'll be searching in such a way that you can easily ignore entire quadrants of data without testing the entire dataset.